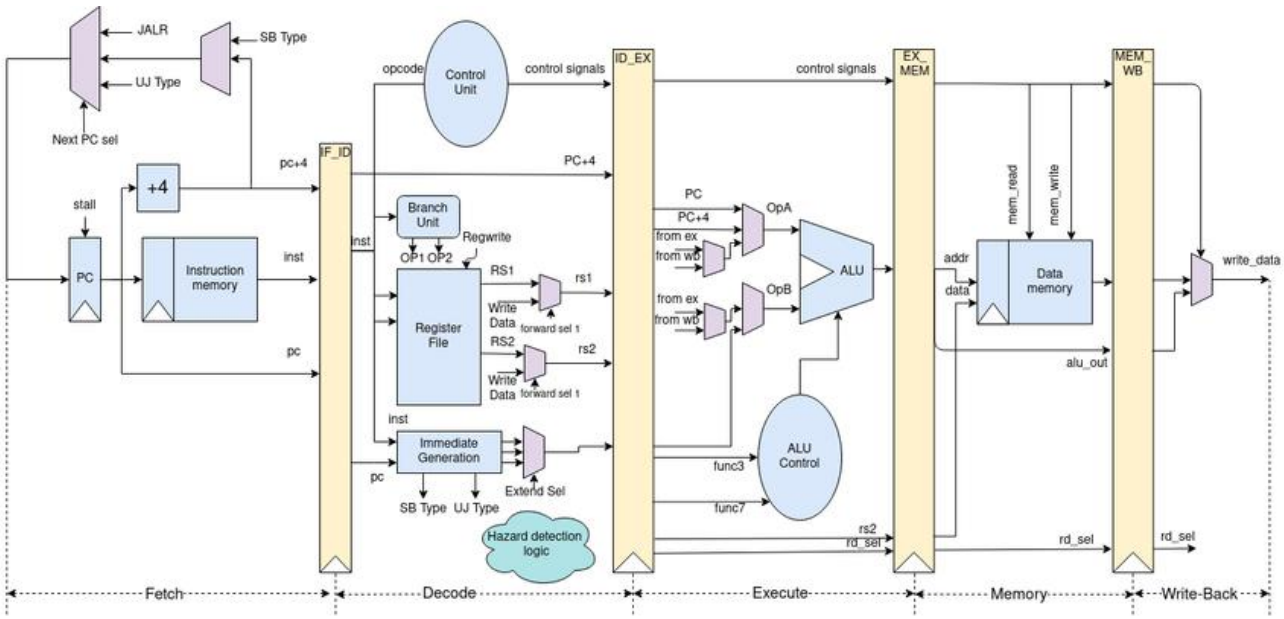
1. KIẾN TRÚC TỔNG THỂ

1.1 Sơ đồ khối tổng thể



Kiến trúc tổng thể của bộ vi xử lý được hiện thực hóa thông qua top module của RISC-V Core. Đây là nơi tích hợp tất cả các thành phần phần cứng từ các tầng thực thi cho đến bộ điều khiển và đơn vị xử lý rủi ro.

Sơ đồ Datapath thể hiện luồng dữ liệu đi xuyên suốt qua 5 tầng đường ống. Bộ điều khiển (Control Unit) nhận mã lệnh từ tầng ID và phát tín hiệu điều khiển đến các bộ chọn (MUX), ALU, và các bộ nhớ dữ liệu. Đơn vị xử lý Hazard (hazard_unit.v) giám sát toàn bộ đường ống để điều phối việc dừng hoặc chuyển tiếp dữ liệu.

1.2 Phân chia đường ống 5 tầng

Để tối ưu hóa hiệu năng và tăng tần số hoạt động (Clock Frequency), thiết kế được chia thành 5 tầng thực thi song song:

- **Tầng Instruction Fetch (IF):** Có nhiệm vụ quản lý địa chỉ lệnh hiện tại thông qua Program Counter (PC) và truy xuất mã lệnh 32-bit từ Instruction Memory (IMEM).
- **Tầng Instruction Decode (ID):** Thực hiện giải mã lệnh để trích xuất các trường dữ liệu. Tại đây, Register File cung cấp giá trị của các thanh ghi nguồn , khối ImmGen tạo ra giá trị tức thời (Immediate) , và Control Unit thiết lập các tín hiệu điều khiển cho các tầng sau.
- **Tầng Execute (EX):** Thực hiện các phép toán số học và logic thông qua đơn vị ALU. Đồng thời, khối Branch Comp thực hiện so sánh toán hạng để phục vụ các lệnh rẽ nhánh.
- **Tầng Memory Access (MEM):** Tương tác với bộ nhớ dữ liệu (DMEM) cho các thao tác đọc/ghi dữ liệu.

- **Tầng Write Back (WB):** Lựa chọn dữ liệu cuối cùng (từ ALU, bộ nhớ hoặc địa chỉ PC+4) để ghi lại vào Register File thông qua khối Load Extension.

1.3 Các thanh ghi đường ống (Pipeline Registers)

Giữa các tầng thực thi là các thanh ghi đường ống đóng vai trò làm bộ đệm để lưu giữ trạng thái dữ liệu và tín hiệu điều khiển của lệnh tại mỗi chu kỳ clock:

- **IF/ID Register**: Lưu trữ PC và mã lệnh để chuyển từ khâu lấy lệnh sang khâu giải mã.
- **ID/EX Register**: Lưu trữ giá trị các thanh ghi nguồn, giá trị tức thời, PC và toàn bộ các tín hiệu điều khiển cho ALU và bộ rẽ nhánh.
- **EX/MEM Register**: Lưu giữ kết quả từ ALU và dữ liệu cần ghi vào bộ nhớ.
- **MEM/WB Register**: Chốt dữ liệu đọc từ bộ nhớ hoặc kết quả ALU để chuẩn bị cho giai đoạn ghi trả về thanh ghi.

Việc sử dụng các thanh ghi này cho phép vi xử lý thực hiện đồng thời tối đa 5 lệnh khác nhau tại mỗi thời điểm, giúp tăng thông lượng xử lý đáng kể so với kiến trúc đơn chu kỳ (Single-cycle).

2. THIẾT KẾ CHI TIẾT CÁC TẦNG DATAPATH

2.1 Tầng Instruction Fetch (IF):

Tầng IF chịu trách nhiệm quản lý luồng địa chỉ và nạp lệnh từ bộ nhớ để đưa vào đường ống.

a) Module 1: Program Counter (pc.v)

- Chức năng: Lưu trữ địa chỉ của lệnh hiện tại đang được thực thi. Cập nhật địa chỉ lệnh tiếp theo (PC Next) vào mỗi sườn lên của xung nhịp đồng hồ (Clock).
- Chiến lược Reset: Sử dụng Reset đồng bộ (Synchronous Reset) tích cực mức thấp. Điều này giúp tránh các lỗi glitch tín hiệu reset không mong muốn, rất phù hợp cho quy trình tổng hợp (synthesis) trên các công cụ hiện đại.
- I/O:

Cổng (Port)	**Chiều (I/O)**	**Kích thước**	**Mô tả chức năng**
clk	I	1 bit	Xung nhịp hệ thống.
rstn	I	1 bit	Tín hiệu reset đồng bộ. Khi rstn = 0, PC sẽ được nạp giá trị khởi tạo (thường là 32'h0000_0000).
pc_en	I	1 bit	Tín hiệu cho phép cập nhật PC (phục vụ cho việc Stall đường ống).
pc_next	I	32 bit	Địa chỉ lệnh tiếp theo, được tính toán từ Datapath (thường là PC + 4 hoặc địa chỉ nhảy từ ALU).
pc_out	O	32 bit	Địa chỉ lệnh hiện tại xuất ra để đưa vào Instruction Memory.

b) Module 2: Instruction Memory (imem.v)

- **Chức năng:** Đóng vai trò là bộ nhớ ROM (Read-Only Memory) của vi xử lý, chuyên dùng để lưu trữ chương trình. Mô-đun này nhận địa chỉ từ khối Program Counter và xuất ra chuỗi lệnh 32-bit tương ứng để đưa vào đường ống giải mã.
- **Đặc tính thiết kế:** Trong cấu trúc kiến trúc Von Neumann hoặc Harvard cơ bản, thao tác đọc lệnh được thiết kế là bất đồng bộ. Điều này đảm bảo ngay khi địa chỉ PC ổn định, mã lệnh sẽ lập tức xuất hiện ở ngõ ra trong cùng một chu kỳ xung nhịp, không làm xuất hiện độ trễ gây đình trệ tầng IF.

Cổng (Port)	**Chiều (I/O)**	**Kích thước**	**Mô tả chức năng**
pc	I	32 bit	Địa chỉ lệnh cần lấy, được cấp từ Program Counter. (Thường bỏ qua 2 bit cuối [1:0] do lệnh RISC-V được căn lề theo Word - Word-aligned).
inst	O	32 bit	Mã lệnh 32-bit tương ứng với địa chỉ đầu vào. Trả về 32'h0 (NOP) nếu địa chỉ nằm ngoài vùng nhớ.

2.2 Tầng Instruction Decode (ID)

Tầng ID thực hiện giải mã lệnh, trích xuất dữ liệu từ tập thanh ghi và tạo giá trị tức thời (Immediate).

a) Module 3: Register File (regfile.v)

- **Chức năng:** Đây là bộ nhớ cục bộ nằm ngay trong CPU, chứa 32 thanh ghi đa dụng (từ x0 đến x31), mỗi thanh ghi rộng 32-bit. Nó có nhiệm vụ cung cấp toán hạng cho ALU và lưu trữ kết quả tính toán hoặc dữ liệu tải về từ bộ nhớ.
- **Ràng buộc đặc biệt của x0:** Trong kiến trúc RISC-V, thanh ghi x0 (zero) được ghim cứng vào giá trị 0. Phần cứng phải đảm bảo: mọi thao tác ghi (write) vào x0 đều bị vô hiệu hóa, và thao tác đọc (read) từ x0 luôn luôn trả về 0.
- **Đặc tính thời gian (Timing):**
  - **Thao tác đọc (Read):** Được thiết kế theo cơ chế bất đồng bộ (asynchronous). Điều này đảm bảo vi xử lý có được dữ liệu ngay lập tức sau khi giải mã xong lệnh, giúp ALU kịp tính toán ngay trong cùng một chu kỳ clock.
  - **Thao tác ghi (Write):** Được thiết kế theo cơ chế đồng bộ (synchronous). Dữ liệu chỉ được chốt (latch) vào thanh ghi đích tại sườn lên của xung nhịp để đảm bảo không xảy ra hiện tượng ghi rác vào bộ nhớ trong lúc tín hiệu đang truyền và chưa ổn định.
- I/O:

Cổng (Port)	**Chiều (I/O)**	**Kích thước**	**Mô tả chức năng**
clk	I	1 bit	Xung nhịp hệ thống.
we	I	1 bit	Tín hiệu cho phép ghi từ Control Unit.
rs1, rs2	I	5 bit	Địa chỉ 2 thanh ghi nguồn cần đọc.
rd	I	5 bit	Địa chỉ thanh ghi đích cần ghi.
wd	I	32 bit	Dữ liệu cần ghi vào.
rd1, rd2	O	32 bit	Dữ liệu xuất ra từ 2 thanh ghi nguồn.

b) Module 4: Immediate generation (imm_gen.v)

- Chức năng:Trích xuất các trường bit chứa giá trị tức thời (Immediate) từ lệnh 32-bit (inst), ghép nối chúng lại, và thực hiện mở rộng dấu (sign-extension) hoặc chèn thêm số 0 (zero-padding) để tạo thành một số nguyên 32-bit hoàn chỉnh (imm_out).
- I/O:

Cổng (Port)	**Chiều (I/O)**	**Kích thước**	**Mô tả chức năng**
inst	I	32 bit	Lệnh gốc từ Instruction Memory [31;7]
imm_sel	I	3 bit	Tín hiệu điều khiển từ Control Unit để chọn định dạng giải mã. Quy ước mã hóa như sau:

			• 3'b000 (I-Type): Cho các lệnh tính toán với hằng số (ADDI, ANDI...), lệnh Load (LW, LB...), và JALR. • 3'b001 (S-Type): Cho các lệnh Store (SW, SB...). • 3'b010 (B-Type): Cho các lệnh Branch (BEQ, BNE...). Chú ý: Lệnh Branch ngầm định bit 0 luôn bằng 0 (dịch trái 1 bit). • 3'b011 (U-Type): Cho lệnh LUI, AUIPC. 12 bit thấp được điền số 0. • 3'b100 (J-Type): Cho lệnh JAL. Tương tự B-Type, bit 0 ngầm định bằng 0.
imm_out	O	32 bit	Giá trị tức thời 32-bit đã được xử lý.

2.3 Tầng Execute (EX)

Tầng EX là trái tim toán học của CPU, xử lý mọi logic tính toán và quyết định rẽ nhánh.

a) Module 5: Arithmetic Logic Unit (alu.v)

- **Chức năng:** Thực hiện phép toán giữa hai toán hạng 32-bit dựa trên mã điều khiển. Khối này hoàn toàn là mạch tổ hợp (Combinational Logic). ALU hỗ trợ đầy đủ các nhóm phép toán: Cộng, Trừ, Logic (AND, OR, XOR), Dịch bit (Trái, Phải logic, Phải số học), và So sánh (Có dấu, Không dấu).
- **Xử lý dấu:** Trong Verilog, các tín hiệu mặc định là không dấu (unsigned). Đối với lệnh dịch phải số học (SRA) và so sánh có dấu (SLT), chúng ta bắt buộc phải ép kiểu bằng hàm hệ thống $signed() để báo cho trình tổng hợp biết cách xử lý bit dấu (bit 31).
- I/O:

Cổng (Port)	**Chiều (I/O)**	**Kích thước**	**Mô tả chức năng**
A	I	1 bit	Toán hạng thứ nhất
B	I	1 bit	Toán hạng thứ hai
alu_sel	I	5 bit	Tín hiệu chọn phép toán từ Control Unit.
out	O	5 bit	Kết quả tính toán.

b) Module 6: Branch Computation (branch_comp.v)

- **Chức năng:** Nhận hai toán hạng từ tập thanh ghi (RegFile) và thực hiện so sánh để phục vụ các lệnh rẽ nhánh (Branch: BEQ, BNE, BLT, BGE, BLTU, BGEU).
  - Lớn hơn hoặc Bằng (BGE): Đơn giản là ~BrLT (Đảo ngược của cờ Nhỏ hơn).
  - Không Bằng (BNE): Đơn giản là ~BrEq (Đảo ngược của cờ Bằng).
  - Control Unit sẽ lo việc lật các tín hiệu này, nên phần cứng của Branch Comp chỉ cần tối giản nhất có thể.

- **Xử lý dấu:** Sử dụng tín hiệu điều khiển BrUn (Branch Unsigned) từ Control Unit để quyết định xem phép so sánh là có dấu (Signed) hay không dấu (Unsigned).

Cổng (Port)	**Chiều (I/O)**	**Kích thước**	**Mô tả chức năng**
rs1, rs2	I	32 bit	Hai dữ liệu cần so sánh (lấy trực tiếp từ rd1 và rd2 của RegFile).
BrUn	I	1 bit	Nếu BrUn = 1, so sánh Unsigned. Nếu BrUn = 0, so sánh Signed.
BrEq	O	1 bit	Bật mức 1 nếu rs1_data == rs2_data.
BrLT	O	1 bit	Bật mức 1 nếu rs1_data < rs2_data.

2.4 Stage MEM:

Module 7: Data Memory (dmem.v)

- **Chức năng:** Là khối bộ nhớ dữ liệu chính (RAM) của hệ thống, thực thi các lệnh nạp dữ liệu (Load) và lưu trữ dữ liệu (Store) từ tập lệnh. Khối này nhận địa chỉ được tính toán từ ALU và dữ liệu cần ghi từ tập thanh ghi (thông qua rs2).
- **Đặc tính thời gian (Timing):**
  - **Thao tác Ghi (Write):** Bắt buộc phải là đồng bộ (Synchronous). Dữ liệu chỉ được cập nhật vào bộ nhớ tại sườn lên của xung nhịp (Clock) khi tín hiệu MemRW ở mức 1. Điều này ngăn chặn việc dữ liệu rác phá hủy bộ nhớ trong quá trình các tín hiệu logic phía trước (ALU) chưa ổn định.
  - **Thao tác Đọc (Read):** Trong kiến trúc Pipeline đơn giản, ngõ đọc thường được thiết kế bất đồng bộ để trả dữ liệu thô (mem_rd_raw) về ngay lập tức.
- **Xử lý kích thước ghi (Store Sizing):** Dựa vào tín hiệu func3 của lệnh Store (SB, SH, SW), mô-đun sẽ phân giải thành các cờ Byte Enable (Masking) để quyết định ghi 1 byte, 2 byte (Half-word) hay 4 byte (Word) vào đúng vị trí bộ nhớ mà không làm ảnh hưởng đến các byte lân cận.

Cổng (Port)	**Chiều (I/O)**	**Kích thước**	**Mô tả chức năng**
clk	I	1 bit	Xung nhịp hệ thống.
MemRW	I	1 bit	Tín hiệu điều khiển: 1 = Ghi bộ nhớ (Store), 0 = Đọc bộ nhớ (Load).
func3	I	3 bit	Trường funct3 của lệnh, dùng để xác định kích thước ghi (000 = SB, 001 = SH, 010 = SW).
addr	I	32 bit	Địa chỉ ô nhớ cần truy xuất (lấy từ ngõ ra của ALU).
wd	I	32 bit	Dữ liệu cần ghi vào bộ nhớ (lấy từ thanh ghi rs2).
rd	O	32 bit	Dữ liệu thô 32-bit (Raw Data) đọc ra từ địa chỉ tương ứng.

2.5 Stage WB:

Module 8: Load Extension (load_extend.v)

- **Chức năng:** Định dạng và chuẩn hóa dữ liệu thô đọc từ Data Memory trước khi lưu trở lại Register File. Vì Data Memory luôn trả về một khối dữ liệu 32-bit, khối Load Extension sẽ làm nhiệm vụ "cắt tỉa" và "kéo giãn" dữ liệu cho đúng với yêu cầu của lệnh Load.
- **Cơ chế hoạt động:**
  - **Định tuyến (Byte/Half-word alignment):** Sử dụng 2 bit thấp nhất của địa chỉ (addr[1:0] từ ALU) để xác định xem byte hoặc half-word cần lấy đang nằm ở vị trí nào trong khối 32-bit thô.
  - **Mở rộng dữ liệu (Data Extension):** Dựa vào tín hiệu func3:
    - **Kéo dài có dấu (Sign-Extension):** Dùng cho LB (Load Byte) và LH (Load Half-word). Mạch sẽ lấy bit dấu (MSB) của byte/half-word vừa trích xuất và nhân bản (replicate) để lấp đầy các bit còn lại của chuẩn 32-bit.
    - **Kéo dài không dấu (Zero-Extension):** Dùng cho LBU (Load Byte Unsigned) và LHU (Load Half-word Unsigned). Mạch chỉ việc chèn thêm các bit 0 vào vị trí còn khuyết.
    - **Bỏ qua (Bypass):** Dùng cho LW (Load Word), giữ nguyên toàn bộ 32-bit.

Cổng (Port)	**Chiều (I/O)**	**Kích thước**	**Mô tả chức năng**
mem_rd_raw	I	32 bit	Dữ liệu thô đọc trực tiếp từ mô-đun Data Memory.
addr_offset	I	2 bit	2 bit thấp nhất của địa chỉ ALU (thường là alu_out[1:0]), dùng để xác định vị trí Offset của byte/half-word.
func3	I	3 bit	Trường funct3 từ lệnh để nhận diện loại lệnh Load (LW, LH, LB, LHU, LBU).
wb_wd	O	32 bit	Dữ liệu đã được chuẩn hóa (căn lề và mở rộng dấu/không dấu) để sẵn sàng ghi vào Register File.

2.6 Registers Between Stages:

3. THIẾT KẾ BỘ ĐIỀU KHIỂN (CONTROL UNIT)

3.1 Chức năng và Sơ đồ giải mã tín hiệu

Chức năng: Khối Control Unit nhận các trường mã lệnh (bao gồm opcode, funct3, funct7) và các cờ trạng thái rẽ nhánh, từ đó xuất ra toàn bộ các tín hiệu điều khiển cần thiết cho Datapath hoạt động. Trong kiến trúc Pipelined, các tín hiệu này được sinh ra tại tầng giải mã (ID) và được truyền dọc theo các thanh ghi đường ống (ID/EX, EX/MEM, MEM/WB) để đồng bộ với dòng chảy của dữ liệu.

I/O:

Cổng (Port)	**Chiều (I/O)**	**Kích thước**	**Mô tả chức năng**
inst	I	32 bit	Lệnh hiện tại. Ta sẽ tự trích xuất opcode, funct3, và funct7[5] bên trong module để code gọn gàng.
BrEq, BrLT	I	1 bit	Các cờ từ khối Branch Comparator để quyết định việc nhảy PC.
PCSel	O	1 bit	0 = PC + 4, 1 = Lấy kết quả từ ALU (dùng cho Branch taken, JAL, JALR).
ImmSel	O	3 bit	Báo cho imm_gen.v biết cách mở rộng giá trị tức thời (I=000, S=001, B=010, U=011, J=100).
RegWEn	O	1 bit	Cho phép ghi vào RegFile.
BrUn	O	1 bit	Báo cho Branch Comp biết phải so sánh có dấu hay không dấu.
ASel	O	1 bit	Cổng A của ALU chọn 0 = rs1_data (từ RegFile), 1 = PC.
BSel	O	1 bit	Cổng B của ALU chọn 0 = rs2_data (từ RegFile), 1 = imm_out (từ ImmGen).
ALUSel	O	4 bit	Phép toán ALU (khớp với bảng mã trong alu.v).
MemRW	O	1 bit	0 = Đọc bộ nhớ, 1 = Ghi bộ nhớ.
WBSel	O	2 bit	Chọn dữ liệu ghi lại (Write-back) vào RegFile: 00 = ALU, 01 = Data Memory, 10 = PC + 4.

Bảng chân lý tập lệnh (Truth Table):

Phân loại	Opcode	PCSel	ImmSel	RegWEn	BrUn	ASel	BSel	ALUSel	MemRW	WBSel
R-Type (ADD, SUB...)	0110011	0	*	1	*	0 (rs1)	0 (rs2)	Theo f3, f7	0	00 (ALU)
I-Type (ADDI, SLLI...)	0010011	0	000 (I)	1	*	0 (rs1)	1 (Imm)	Theo f3, f7	0	00 (ALU)
Load (LW, LB...)	0000011	0	000 (I)	1	*	0 (rs1)	1 (Imm)	0000 (ADD)	0	01 (Mem)
Store (SW, SB...)	0100011	0	001 (S)	0	*	0 (rs1)	1 (Imm)	0000 (ADD)	1	*
Branch (BEQ, BLT...)	1100011	*(Tùy cờ)*	010 (B)	0	*(Tùy f3)*	1 (PC)	1 (Imm)	0000 (ADD)	0	*
JAL	1101111	1	100 (J)	1	*	1 (PC)	1 (Imm)	0000 (ADD)	0	10 (PC+4)
JALR	1100111	1	000 (I)	1	*	0 (rs1)	1 (Imm)	0000 (ADD)	0	10 (PC+4)
LUI	0110111	0	011 (U)	1	*	*	1 (Imm)	1010 (PASSB)	0	00 (ALU)
AUIPC	0010111	0	011 (U)	1	*	1 (PC)	1 (Imm)	0000 (ADD)	0	00 (ALU)

- **Nhóm lệnh Rẽ nhánh (Branch):** ALU được cấu hình làm nhiệm vụ cộng địa chỉ (ASel = 1 lấy PC, BSel = 1 lấy Immediate, ALUSel = ADD) để tạo ra địa chỉ nhảy tiềm năng (Branch Target). Tuy nhiên, tín hiệu PCSel chỉ bằng 1 nếu các cờ BrEq hoặc BrLT thỏa mãn điều kiện funct3.
- **Nhóm lệnh Lưu trữ (Store):** Do không cần ghi kết quả trả về thanh ghi nên cờ RegWEn = 0. ALU thực hiện cộng rs1 và Immediate để tạo địa chỉ bộ nhớ. Lúc này, tín hiệu WBSel trở thành "Don't care" (*) vì dù có chọn dữ liệu nào thì cũng không được ghi vào RegFile.
- **Lệnh JAL và JALR:** Tín hiệu PCSel luôn bị ép bằng 1 để buộc bộ đếm chương trình rẽ nhánh. Tín hiệu WBSel = 10 để chọn đường truyền giá trị PC + 4 lưu vào thanh ghi đích, phục vụ việc lưu địa chỉ quay về sau khi thực hiện xong hàm (Function Call).

4. KẾT QUẢ MÔ PHỎNG VÀ ĐÁNH GIÁ (VERIFICATION)

4.1 Phương pháp kiểm thử

Do phiên bản này được thiết kế thuần túy tập trung vào việc định tuyến luồng dữ liệu (Datapath) và giải mã (Control Unit) mà chưa tích hợp khối xử lý rủi ro phần cứng (Hazard Unit), các hiện tượng Data Hazard và Control Hazard sẽ xảy ra nếu các lệnh phụ thuộc được xếp sát nhau.

Để giải quyết vấn đề này và đảm bảo CPU hoạt động đúng, phương pháp **Lập lịch bằng phần mềm (Software Scheduling)** được áp dụng. Cụ thể:

- **Xử lý Data Hazard:** Trình biên dịch (hoặc người viết mã máy) sẽ chèn thủ công **3 lệnh NOP** (ADDI x0, x0, 0 - Mã Hex: 0x00000013) vào giữa lệnh sinh dữ liệu (Producer) và lệnh sử dụng dữ liệu (Consumer). Khoảng trễ 3 chu kỳ này đảm bảo lệnh Producer đã đi đến tầng Write Back (WB) và ghi xong vào Register File trước khi lệnh Consumer tiến hành đọc tại tầng Decode (ID).
- **Xử lý Control Hazard:** Tương tự, 3 lệnh NOP được chèn ngay sau các lệnh rẽ nhánh (Branch) và lệnh nhảy (Jump) để lấp đầy đường ống trong lúc chờ CPU tính toán xong địa chỉ đích tại tầng EX, tránh hiện tượng thực thi nhầm các lệnh rác.

4.2 Cấu trúc Môi trường kiểm thử tự động

Để kiểm tra độ chính xác của 37 lệnh cơ bản mà không cần phải quan sát thủ công từng chu kỳ xung nhịp trên giản đồ sóng (Waveform), một Testbench tự động (tb_riscv_checker.sv) đã được xây dựng với kiến trúc như sau:

1. **Bộ nhớ lệnh (Instruction Memory Initialization):** Sử dụng lệnh hệ thống $readmemh() của Verilog để nạp tệp tin chứa 37 lệnh mã máy (đã chèn sẵn các lệnh NOP) vào mảng bộ nhớ của mô-đun imem.v.
2. **Expected Results:** Khởi tạo một mảng lưu trữ các giá trị đúng (Expected Values) đã được tính toán trước cho các thanh ghi. Mảng này chỉ theo dõi các lệnh có thao tác ghi trả kết quả về Register File (không bao gồm lệnh Store và Branch).
3. **Bộ giám sát (Monitor / Checker):** Sử dụng một khối always @(negedge clk) để liên tục giám sát tầng Write Back. Cứ mỗi khi tín hiệu wb_RegWEn được bật và thanh ghi đích không phải là x0, bộ giám sát sẽ:
    - Trích xuất dữ liệu thực tế (wb_wd) từ CPU.
    - So sánh với dữ liệu mong đợi trong mảng expected_results.
    - In ra cửa sổ Transcript kết quả [PASS] (nếu khớp) hoặc [FAIL] (nếu sai lệch), đồng thời đếm tổng số lệnh vượt qua bài test.

Việc lấy mẫu tại sườn xuống (negedge clk) giúp đảm bảo dữ liệu tại các mạch tổ hợp đã hoàn toàn ổn định sau sườn lên, loại bỏ nguy cơ đọc sai do nhiễu tín hiệu (race condition).

4.3 Kịch bản kiểm thử (Test Scenarios)

Tệp tin mã máy được thiết kế để kiểm tra bao quát toàn bộ các nhóm lệnh cấu thành nên RV32I:

- **Nhóm U-Type (LUI, AUIPC):** Kiểm tra khả năng nạp hằng số lớn và làm việc với Program Counter.
- **Nhóm I-Type & R-Type ALU:** Kiểm tra khả năng tính toán của khối ALU với đầy đủ các phép Cộng, Trừ, Logic (AND, OR, XOR), Dịch bit, và So sánh (SLT, SLTU).
- **Nhóm Memory (Load/Store):** Khởi tạo việc ghi các kích thước dữ liệu khác nhau (Byte, Half-word, Word) vào Data Memory, sau đó dùng các lệnh Load tương ứng để đọc ngược lại, nhằm kiểm tra khối Load Extension.
- **Nhóm Control Flow (Branch/Jump):** Kiểm tra các lệnh JAL, JALR và đầy đủ 6 điều kiện rẽ nhánh (BEQ, BNE, BLT, BGE, BLTU, BGEU). Đảm bảo CPU ghi đúng địa chỉ quay về (PC + 4) vào thanh ghi đích và rẽ nhánh chính xác đến địa chỉ mục tiêu.

STT	Mã Hex (32-bit)	Lệnh Assembly tương đương	Nhóm lệnh	Chức năng & Kết quả dự kiến trong Testbench
1	123450B7	LUI x1, 0x12345	**U-Type**	Tải hằng số 20-bit vào thanh ghi. **x1 = 0x12345000**
2	00001117	AUIPC x2, 1	**U-Type**	Cộng hằng số vào PC hiện tại. **x2 = PC + 0x1000**
3	00F00193	ADDI x3, x0, 15	**I-Type**	Cộng hằng số. **x3 = 15** (Dùng làm toán hạng gốc)
4	0141A213	SLTI x4, x3, 20	**I-Type**	So sánh nhỏ hơn (Có dấu). Vì 15 < 20 → **x4 = 1**
5	FFF1B293	SLTIU x5, x3, -1	**I-Type**	So sánh nhỏ hơn (Không dấu). 15 < 0xFFFFFFFF → **x5 = 1**
6	00F1C313	XORI x6, x3, 15	**I-Type**	Phép XOR logic. 15 ^ 15 → **x6 = 0**
7	00A06393	ORI x7, x0, 10	**I-Type**	Phép OR logic. **x7 = 10** (Dùng làm toán hạng gốc)
8	00F3F413	ANDI x8, x7, 15	**I-Type**	Phép AND logic. 10 & 15 → **x8 = 10**
9	00239493	SLLI x9, x7, 2	**I-Type**	Dịch trái logic. 10 << 2 → **x9 = 40**
10	0014D513	SRLI x10, x9, 1	**I-Type**	Dịch phải logic. 40 >> 1 → **x10 = 20**
11	40155593	SRAI x11, x10, 1	**I-Type**	Dịch phải số học (giữ bit dấu). 20 >> 1 → **x11 = 10**
12	00338633	ADD x12, x7, x3	**R-Type**	Phép cộng. 10 + 15 → **x12 = 25**
13	407186B3	SUB x13, x3, x7	**R-Type**	Phép trừ. 15 - 10 → **x13 = 5**
14	00D39733	SLL x14, x7, x13	**R-Type**	Dịch trái theo thanh ghi. 10 << 5 → **x14 = 320**

15	0033A7B3	SLT x15, x7, x3	**R-Type**	So sánh nhỏ hơn. 10 < 15 → **x15 = 1**
16	0076B833	SLTU x16, x13, x7	**R-Type**	So sánh nhỏ hơn không dấu. 5 < 10 → **x16 = 1**
17	0033C8B3	XOR x17, x7, x3	**R-Type**	Phép XOR thanh ghi. 10 ^ 15 → **x17 = 5**
18	00D75933	SRL x18, x14, x13	**R-Type**	Dịch phải logic theo thanh ghi. 320 >> 5 → **x18 = 10**
19	40D759B3	SRA x19, x14, x13	**R-Type**	Dịch phải số học theo thanh ghi. 320 >> 5 → **x19 = 10**
20	0033EA33	OR x20, x7, x3	**R-Type**	Phép OR thanh ghi. 10 | 15 → **x20 = 15**
21	0033FAB3	AND x21, x7, x3	**R-Type**	Phép AND thanh ghi. 10 & 15 → **x21 = 10**
22	00302023	SW x3, 0(x0)	**S-Type**	Lưu Word (32-bit). Ghi giá trị 15 vào bộ nhớ tại địa chỉ 0.
23	00701223	SH x7, 4(x0)	**S-Type**	Lưu Halfword (16-bit). Ghi giá trị 10 vào địa chỉ 4.
24	00D00323	SB x13, 6(x0)	**S-Type**	Lưu Byte (8-bit). Ghi giá trị 5 vào địa chỉ 6.
25	00002B03	LW x22, 0(x0)	**I-Type**	Đọc Word (32-bit). Đọc địa chỉ 0 → **x22 = 15**
26	00401B83	LH x23, 4(x0)	**I-Type**	Đọc Halfword (Kéo dài dấu). Đọc địa chỉ 4 → **x23 = 10**
27	00600C03	LB x24, 6(x0)	**I-Type**	Đọc Byte (Kéo dài dấu). Đọc địa chỉ 6 → **x24 = 5**
28	00405C83	LHU x25, 4(x0)	**I-Type**	Đọc Halfword (Chèn số 0). Đọc địa chỉ 4 → **x25 = 10**
29	00604D03	LBU x26, 6(x0)	**I-Type**	Đọc Byte (Chèn số 0). Đọc địa chỉ 6 → **x26 = 5**
30	01000DEF	JAL x27, +16	**J-Type**	Nhảy vô điều kiện (+16 byte), bỏ qua các lệnh NOP phía sau. Lưu PC+4 vào x27.
31	00000E97	AUIPC x29, 0	**U-Type**	Lệnh phụ trợ: Lưu địa chỉ lệnh hiện tại vào x29 để phục vụ lệnh nhảy JALR ngay bên dưới.
32	020E8E67	JALR x28, x29, 32	**I-Type**	Nhảy tới địa chỉ (x29 + 32). Thoát khỏi vòng lặp và đi tiếp. Lưu PC+4 vào x28.

33	00738863	BEQ x7, x7, +16	**B-Type**	Nhánh Bằng nhau. (10 == 10) → Nhảy +16 byte.
34	00379863	BNE x7, x3, +16	**B-Type**	Nhánh Không Bằng. (10 != 15) → Nhảy +16 byte.
35	0037C863	BLT x7, x3, +16	**B-Type**	Nhánh Nhỏ hơn. (10 < 15) → Nhảy +16 byte.
36	0071D863	BGE x3, x7, +16	**B-Type**	Nhánh Lớn hơn hoặc Bằng. (15 >= 10) → Nhảy +16 byte.
37	0037E863	BLTU x7, x3, +16	**B-Type**	Nhánh Nhỏ hơn (Không dấu). (10 < 15) → Nhảy +16 byte.
38	0071F863	BGEU x3, x7, +16	**B-Type**	Nhánh Lớn hơn hoặc Bằng (Không dấu). (15 >= 10) → Nhảy +16 byte.

Mặc dù chương trình có 38 lệnh thực thi, bộ Automatic Checker báo cáo 29 lệnh. Điều này hoàn toàn đúng với bản chất vi kiến trúc, cụ thể:

- **29 lệnh được Check:** Là những lệnh có thao tác ghi kết quả trả về Register File (RegWEn = 1). Bao gồm các lệnh tính toán ALU (R-Type, I-Type), nạp hằng số (U-Type), tải dữ liệu (Load), và lưu địa chỉ nhảy (JAL, JALR).
- **9 lệnh KHÔNG được Check trực tiếp:** Bao gồm 3 lệnh Store (SW, SH, SB) và 6 lệnh Branch (BEQ, BNE, BLT, BGE, BLTU, BGEU). Các lệnh này chỉ làm thay đổi trạng thái Data Memory hoặc Program Counter mà không ghi vào Register File (RegWEn = 0), do đó bộ Monitor bỏ qua chúng. Các lệnh này được **kiểm chứng gián tiếp (Implicit Verification)** một cách chắc chắn:
    - Các lệnh Load đọc ra đúng dữ liệu chứng tỏ các lệnh Store trước đó đã ghi chính xác vào bộ nhớ.
    - Chương trình chạy trơn tru đến dòng cuối cùng và bỏ qua được các lệnh rác chứng tỏ 6 lệnh Branch đã tính toán đúng điều kiện và điều hướng PC rẽ nhánh thành công.

4.4 Kết quả mô phỏng và đánh giá

a) Kết quả mô phỏng tự động (Transcript Log)

```
# ----------------------------------------------------------------
#                  STARTING RISC-V SIMULATION
# ----------------------------------------------------------------
# [PASS] [U-Type] LUI   (Check  0) : Reg x1  successfully updated to 0x12345000
# [PASS] [U-Type] AUIPC (Check  1) : Reg x2  successfully updated to 0x00001010
# [PASS] [I-Type] ADDI  (Check  2) : Reg x3  successfully updated to 0x0000000f
# [PASS] [I-Type] SLTI  (Check  3) : Reg x4  successfully updated to 0x00000001
# [PASS] [I-Type] SLTIU (Check  4) : Reg x5  successfully updated to 0x00000001
# [PASS] [I-Type] XORI  (Check  5) : Reg x6  successfully updated to 0x00000000
# [PASS] [I-Type] ORI   (Check  6) : Reg x7  successfully updated to 0x0000000a
# [PASS] [I-Type] ANDI  (Check  7) : Reg x8  successfully updated to 0x0000000a
# [PASS] [I-Type] SLLI  (Check  8) : Reg x9  successfully updated to 0x00000028
# [PASS] [I-Type] SRLI  (Check  9) : Reg x10 successfully updated to 0x00000014
# [PASS] [I-Type] SRAI  (Check 10) : Reg x11 successfully updated to 0x0000000a
# [PASS] [R-Type] ADD   (Check 11) : Reg x12 successfully updated to 0x00000019
# [PASS] [R-Type] SUB   (Check 12) : Reg x13 successfully updated to 0x00000005
# [PASS] [R-Type] SLL   (Check 13) : Reg x14 successfully updated to 0x00000140
# [PASS] [R-Type] SLT   (Check 14) : Reg x15 successfully updated to 0x00000001
# [PASS] [R-Type] SLTU  (Check 15) : Reg x16 successfully updated to 0x00000001
# [PASS] [R-Type] XOR   (Check 16) : Reg x17 successfully updated to 0x00000005
# [PASS] [R-Type] SRL   (Check 17) : Reg x18 successfully updated to 0x0000000a
# [PASS] [R-Type] SRA   (Check 18) : Reg x19 successfully updated to 0x0000000a
# [PASS] [R-Type] OR    (Check 19) : Reg x20 successfully updated to 0x0000000f
# [PASS] [R-Type] AND   (Check 20) : Reg x21 successfully updated to 0x0000000a
# [PASS] [Load  ] LW    (Check 21) : Reg x22 successfully updated to 0x0000000f
# [PASS] [Load  ] LH    (Check 22) : Reg x23 successfully updated to 0x0000000a
# [PASS] [Load  ] LB    (Check 23) : Reg x24 successfully updated to 0x00000005
# [PASS] [Load  ] LHU   (Check 24) : Reg x25 successfully updated to 0x0000000a
# [PASS] [Load  ] LBU   (Check 25) : Reg x26 successfully updated to 0x00000005
# [PASS] [Jump  ] JAL   (Check 26) : Reg x27 successfully updated to 0x00000108
# [PASS] [Jump  ] AUIPC (Check 27) : Reg x29 successfully updated to 0x00000114
# [PASS] [Jump  ] JALR  (Check 28) : Reg x28 successfully updated to 0x00000128
#
# ================================================================
#                     FINAL SIMULATION REPORT
# ================================================================
#  Total writes checked : 29 / 29
#  Passed               : 29
#  Failed               : 0
# ================================================================
```

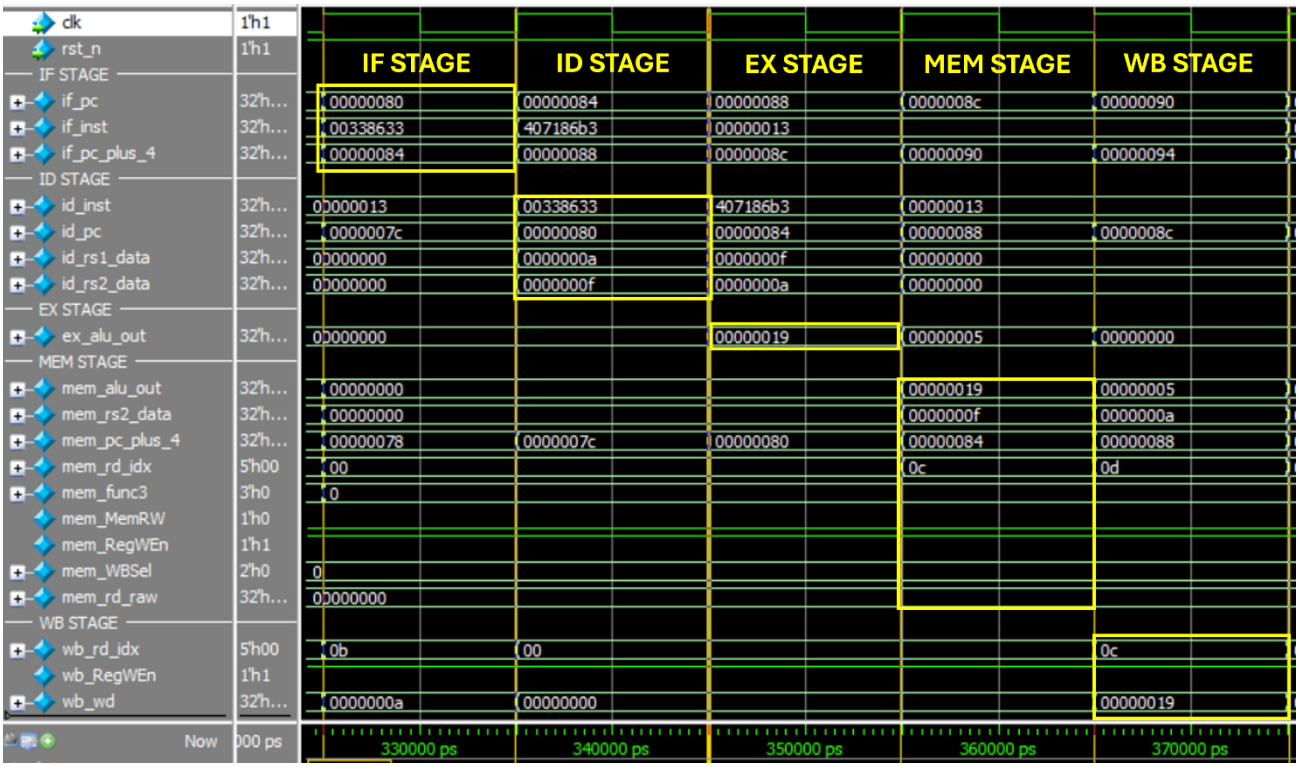
Quá trình chạy mô phỏng cho kết quả đúng với tất cả các lệnh. Kết quả "Total writes checked: 29 / 29 | Failed: 0" khẳng định bộ điều khiển (Control Unit) đã giải mã chính xác các trường opcode, funct3, funct7 và phần cứng Datapath đã tính toán hoàn toàn đúng.

b) Phân tích Giản đồ thời gian (Waveform Analysis)

Vòng đời của một lệnh qua 5 tầng (Datapath Flow):

Lấy ví dụ lệnh ADD x12, x7, x3 (mã HEX: 0338633):

- **Thực thi:** Reg[x12] = Reg[x7] + Reg[x3]
- **Ý nghĩa:** Lấy giá trị đang lưu trữ trong thanh ghi x7 cộng với giá trị trong thanh ghi x3, sau đó ghi kết quả vào thanh ghi x12



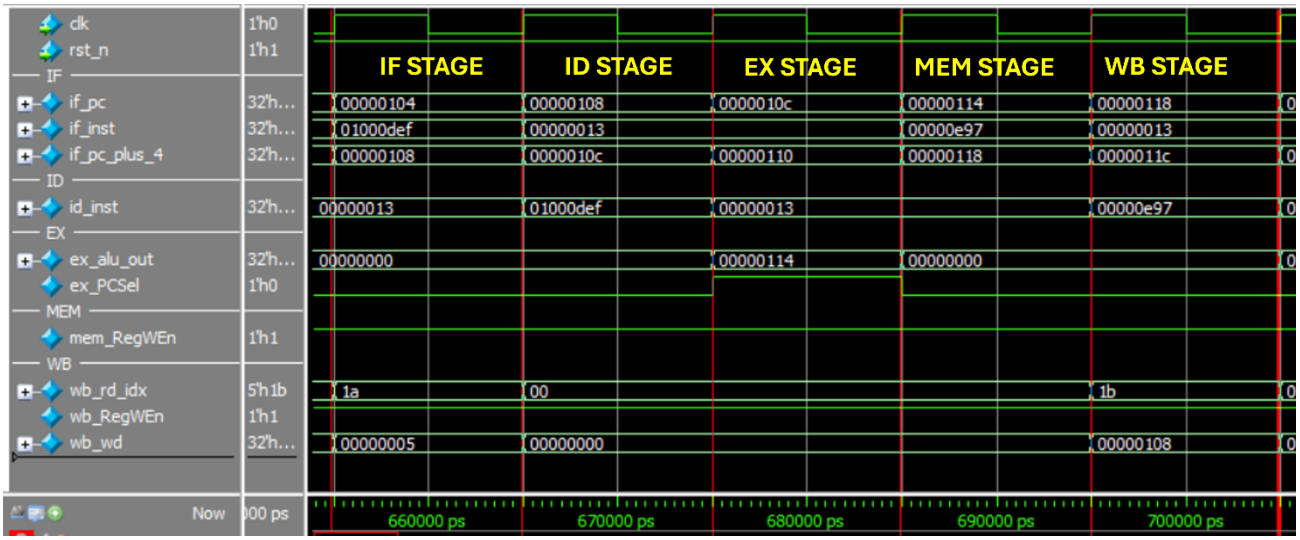
Như trên giản đồ xung, lệnh này "trôi" qua các tầng như sau:

- **IF (Instruction Fetch):** CPU dùng địa chỉ từ PC (ví dụ 0x80) để lấy mã lệnh 0x00338633 từ Instruction Memory.
- **ID (Instruction Decode):** Control Unit nhận diện opcode 0x33 và xác lập các tín hiệu điều khiển (RegWEn = 1, ALUSel = ADD).
    - Register File đọc giá trị từ x7 (ví dụ 10) và x3 (ví dụ 15).
- **EX (Execute):** ALU thực hiện phép cộng: 10 + 15 = 25 (0x19). Đây là thời điểm kết quả 0x19 xuất hiện tại ngõ ra của ALU.
- **MEM (Memory):** Vì đây là lệnh ALU, không phải lệnh Load/Store, nên tầng này không thực hiện thao tác với bộ nhớ. Giá trị 25 đơn giản được chuyển tiếp qua tầng này.
- **WB (Write Back):** Tại sườn xuống của xung nhịp, giá trị 25 chính thức được chốt và ghi vào thanh ghi x12 trong Register File.

Quá trình rẽ nhánh (Branching):

Ví dụ lệnh JAL x27, +16

- **Thực thi:** Reg[x27] = PC + 4 và PC = PC + 16
- **Ý nghĩa:** Lưu địa chỉ của lệnh kế tiếp (PC + 4) vào thanh ghi liên kết x27, sau đó thực hiện bước nhảy (cập nhật PC) đến địa chỉ đích nằm cách PC hiện tại 16 byte.



Lệnh được thực hiện qua các tầng như sau:

- **IF (Instruction Fetch):** CPU dùng địa chỉ từ PC (0x104) để lấy mã lệnh 0x01000DEF từ Instruction Memory.
- **ID (Instruction Decode):** Control Unit nhận diện opcode 0x6F (của nhóm lệnh JAL) và xác lập các tín hiệu điều khiển (RegWEn = 1, PCSel = 1).
    - Đồng thời, khối ImmGen trích xuất và mở rộng mã lệnh để tạo ra giá trị tức thời (Immediate) là +16.
- **EX (Execute):** ALU tiến hành tính toán địa chỉ đích nhảy: 0x104 + 16 (0x10) = 0x114. Tín hiệu PCSel = 1 sẽ lập tức ép bộ chọn tại tầng IF cập nhật PC mới thành 0x114 cho chu kỳ tiếp theo. Đồng thời, giá trị địa chỉ quay về (PC + 4 = 0x108) được tính toán để chuẩn bị truyền đi tiếp.
- **MEM (Memory):** Vì JAL không phải là lệnh Load/Store, tầng này hoàn toàn không truy xuất bộ nhớ. Giá trị địa chỉ quay về (0x108) đơn giản được chuyển tiếp qua thanh ghi của tầng này.
- **WB (Write Back):** Tại sườn xuống của xung nhịp, giá trị 0x108 chính thức được chốt và ghi vào thanh ghi x27 (địa chỉ 27 hay 0x1B) trong Register File để hoàn tất việc lưu địa chỉ liên kết.